

Lab0 实验报告

一、思考题

1、Git的使用 1

- 在前述已初始化的 ~/learnGit 目录下，创建一个名为 README.txt 的文件。执行命令 `git status > Untracked.txt`（其中的 `>` 为输出重定向，我们将在 0.6.3 中详细介绍）。

```
cd ~/learnGit
touch README.txt
git status > Untracked.txt
```

- 在 README.txt 文件中添加任意文件内容，然后使用 `add` 命令，再执行命令 `git status > Stage.txt`。

```
vim README.txt
# todo in the txt
git add README.txt
git status > Stage.txt
```

- 提交 README.txt，并在提交说明里写入自己的学号。

```
git commit -m "23371325"
```

- 执行命令 `cat Untracked.txt` 和 `cat Stage.txt`，对比两次运行的结果，体会 README.txt 两次所处位置的不同。

```
cat Untracked.txt
cat Stage.txt
```

Untracked.txt 会显示 README.txt 是未跟踪的文件；
而 Stage.txt 会显示 README.txt 已经被添加到暂存区，准备提交。
第一次处于 Untraced 未跟踪状态，第二次处于 Staged 已暂存状态

- 修改 README.txt 文件，再执行命令 `git status > Modified.txt`。

```
vim README.txt
# todo in the txt
git status > Modified.txt
```

- 执行命令 `cat Modified.txt`，观察其结果和第一次执行 `add` 命令之前的 `status` 是否一样，并思考原因。

```
cat Modified.txt
```

Modified.txt 会显示 README.txt 已经被修改，但尚未添加到暂存区。
这与第一次执行 `add` 之前的状态并不完全一样，因为文件处于 Modified 已修改的状态。

2、箭头与命令

- 仔细看看0.10，思考一下箭头中的 add the file、stage the file 和 commit 分别对应的是 Git 里的哪些命令呢？

```
git add test.txt # add the file
git add test.txt # stage the file
git commit -m "something" # commit
```

3、Git的一些场景

- 代码文件 print.c 被错误删除时，应当使用什么命令将其恢复？

```
git checkout -- print.c
```

- 代码文件 print.c 被错误删除后，执行了 git rm print.c 命令，此时应当使用什么命令将其恢复？

```
git reset HEAD print.c
git checkout -- print.c
```

- 无关文件 hello.txt 已经被添加到暂存区时，如何在不删除此文件的前提下将其移出暂存区？

```
git restore hello.txt
```

4、Git的使用 2

- 找到在 /home/22xxxxxx/learnGit 下刚刚创建的 README.txt 文件，若不存在则新建该文件。

```
cd /home/23371325/learnGit
touch README.txt
```

- 在文件里加入 Testing 1，git add，git commit，提交说明记为 1。

```
echo "Testing 1" > README.txt
git add README.txt
git commit -m "1"
```

- 模仿上述做法，把 1 分别改为 2 和 3，再提交两次。

```
echo "Testing 2" > README.txt
git add README.txt
git commit -m "2"
echo "Testing 3" > README.txt
git add README.txt
git commit -m "3"
```

- 使用 `git log` 命令查看提交日志，看是否已经有三次提交，记下提交说明为3的哈希值a。

```
git log
```

```
git@23371325:~/learnGit (master)$ git log
commit 0de385113f631282d6856753338c213e023ea3af (HEAD -> master)
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:55 2025 +0800

    3

commit 89cdcd587d7b1b2a7291c9672f30e2eaf71d58f
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:55 2025 +0800

    2

commit 710e9606a9dc1c5fc7c9210b03dd28515c88ede8
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:39 2025 +0800

    1

commit 89d41108666f4ec3da8d8811d54baf66e18df542
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:00:54 2025 +0800

    23371325
```

说明为3的哈希值a如图所示

- 进行版本回退。执行命令 `git reset --hard HEAD^` 后，再执行 `git log`，观察其变化。

```
git reset --hard HEAD^
git log
```

```

git@23371325:~/learnGit (master)$ git reset --hard HEAD^
git log
HEAD 现在位于 89cdcdd 2
commit 89cdcdd587d7b1b2a7291c9672f30e2eaf71d58f (HEAD -> master)
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:55 2025 +0800

    2

commit 710e9606a9dc1c5fc7c9210b03dd28515c88ede8
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:39 2025 +0800

    1

commit 89d41108666f4ec3da8d8811d54baf66e18df542
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:00:54 2025 +0800

    23371325

```

提交说明为3的记录消失

- 找到提交说明为 1 的哈希值，执行命令 `git reset --hard` 后，再执行 `git log`，观察其变化。

```

git reset --hard 710e9606a9dc1c5fc7c9210b03dd28515c88ede8
git log

```

```

git@23371325:~/learnGit (master)$ git reset --hard 710e9606a9dc1c5fc7c9210b03dd28515c88ede8
git log
HEAD 现在位于 710e960 1
commit 710e9606a9dc1c5fc7c9210b03dd28515c88ede8 (HEAD -> master)
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:39 2025 +0800

    1

commit 89d41108666f4ec3da8d8811d54baf66e18df542
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:00:54 2025 +0800

    23371325

```

`git log` 的输出此时只显示提交说明为 1 的记录。

- 现在已经回到了旧版本，为了再次回到新版本，执行 `git reset --hard`，再执行 `git log`，观察其变化。

```

git reset --hard 0de385113f631282d6856753338c213e023ea3af
git log

```

```
git@23371325:~/learnGit (master)$ git reset --hard 0de385113f631282d6856753338c213e023ea3a
git log
HEAD 现在位于 0de3851 3
commit 0de385113f631282d6856753338c213e023ea3af (HEAD -> master)
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:55 2025 +0800

    3

commit 89cdcd587d7b1b2a7291c9672f30e2eaf71d58f
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:55 2025 +0800

    2

commit 710e9606a9dc1c5fc7c9210b03dd28515c88ede8
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:25:39 2025 +0800

    1

commit 89d41108666f4ec3da8d8811d54baf66e18df542
Author: 吴思远 <23371325@buaa.edu.cn>
Date: Tue Mar 11 09:00:54 2025 +0800

    23371325
```

git add 的输出恢复到了提交说明为3的记录

5、echo 的使用

- 执行如下命令, 并查看结果

```
echo first
```

```
echo second > output.txt
```

```
echo third > output.txt
```

```
echo forth >> output.txt
```

第一条命令在终端输出first;

第二条命令会将 second 写入到 output.txt 文件中;

第三条命令会将 third 写入到 output.txt 文件中, 并覆盖之前的内容;

第四条命令会将 forth 追加到 output.txt 文件的末尾, 但是不会覆盖之前的内容。

6、文件的操作

- 使用你知道的方法 (包括重定向) 创建下图内容的文件 (文件命名为 test), 将创建该文件的命令序列保存在 command 文件中, 并将 test 文件作为批处理文件运行, 将运行结果输出至 result 文件中。给出 command 文件和 result 文件的内容, 并对最后的结果进行解释说明 (可以从 test 文件的内容入手)。具体实现的过程中思考下列问题: echo Shell Start 与 echo echo Shell Start 效果是否有区别; echo echo \$c>file1 与 echo echo \$c>file1 效果是否有区别。

```
# command文件的命令序列
echo 'echo Shell Start...' > test
echo 'echo set a = 1' >> test
echo 'a=1' >> test
echo 'echo set b = 2' >> test
echo 'b=2' >> test
```

```
echo 'echo set c = a+b' >> test
echo 'c=${a+$b}' >> test
echo 'echo c = $c' >> test
echo 'echo save c to ./file1' >> test
echo 'echo $c>file1' >> test
echo 'echo save b to ./file2' >> test
echo 'echo $b>file2' >> test
echo 'echo save a to ./file3' >> test
echo 'echo $a>file3' >> test
echo 'echo save file1 file2 file3 to file4' >> test
echo 'cat file1>file4' >> test
echo 'cat file2>>file4' >> test
echo 'cat file3>>file4' >> test
echo 'echo save file4 to ./result' >> test
echo 'cat file4>>result' >> test
```

```
bash test > result
```

```
# result文件内容
Shell Start...
set a = 1
set b = 2
set c = a+b
c = 3
save c to ./file1
save b to ./file2
save a to ./file3
save file1 file2 file3 to file4
save file4 to ./result
```

前者均会输出echo后面的一段字符串到终端，而后者会先执行反引号里的内容，让标准输出结果作为参数传递给echo

二、难点分析

Lab0第三道练习题要求通过bash命令生成result文件，文件内容是某个文件中某个字符串所在行数。

其中查找字符串可以用grep指令，但是grep指令会输出包含该字符串的行号和内容，所以我们还需要使用

cut指令单独提取行号的部分，并且重定向cut的输出到目标文件，再用管道将grep和cut连接起来

Lab0第四题替换fibonacci文件的指定字符串采用 `sed -i 's/<string1>/<string2>/g'` 即可解决而之后补全Makefile文件与前面几题不同，需要补全两个不同目录下的Makefile文件，且两者之间存在依赖关系，

所以我们应该使用 `make -C ./code` 执行子目录的Makefile文件。同时在fibonacci.c与main.c原文件中fibonacci.h文件是用尖括号括起的，且该文件位于csc/include目录下，所以gcc在编译时只会从/usr/include下寻找

无法找到fibonacci.h文件，需要在gcc后加上 `-I dir` 来增加标准包含目录。其余的操作与前面的练习题大同小异

三、实验体会

在Lab0的实验对假期的预习内容进行了一定程度的考察，同时还让我逐渐熟悉了Linux系统的命令行操作模式和一些指令，也体会到一些CLI Shell实现某些操作的简便快捷，如练习题2中对一部分特定文件的批量重命名，练习题3获取目标文件含有特定字符串的行数。

从一开始面对黑漆漆一块屏幕和不断闪烁的光标的不知所措，经过假期的预习和Lab0的趁热打铁，到现在逐渐适应CLI的操作模式，对未来一系列接踵而至的实验也少了些迷惘与畏惧。希望这学期的操作系统实验课能取得一个让自己满意的好成绩！

四、原创说明

- 本文部分内容在LLM的协助下完成